CS255 Mini Project

Simulation-based Performance Analysis of an NR-U and WiFi Coexistence System in the Presence of Hidden Nodes

Submitted by

Aadharsh Venkat - 241CS101
Amisha Vidya Diwakar - 241CS107

Department of Computer Science and Engineering

National Institute of Technology Karnataka (NITK)

Date: May 7, 2026

Contents

1. Architecture

1.1 System Architecture Overview

The proposed system follows a multi-layer simulation and analysis architecture to evaluate the coexistence performance of NR-U and WiFi networks in the presence of hidden nodes. The architecture integrates network modeling, packet-level simulation, data capture, and performance analysis into a unified workflow.

The network topology consists of a two-dimensional spatial region where nodes are randomly distributed. The distribution follows a homogeneous Poisson Point Process (PPP), which realistically models wireless deployments in unlicensed spectrum environments.

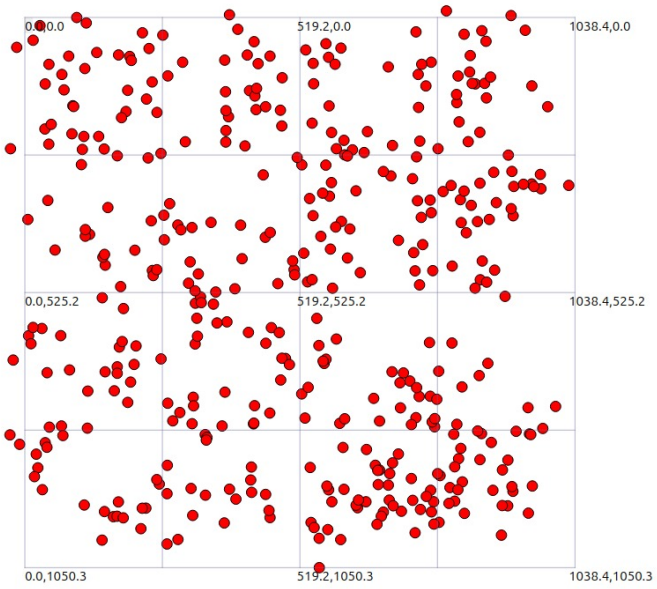

Figure 1: Random spatial distribution of nodes in the simulation area

1.2 Network Topology Layer

The network topology forms the foundation of the system.

Key Characteristics:

- Nodes are randomly distributed over a bounded 2D area.
- The simulation area is divided into a grid for visualization.
- Distribution follows a homogeneous Poisson Point Process (PPP).

Node Types:

- NR-U gNBs (5G base stations)
- WiFi Access Points (APs)
- User devices (receivers)

Spatial Behavior: Each node is associated with:

- Transmission range
- Sensing range

Based on these ranges:

- **Sensible nodes:** Nodes that can detect transmissions
- **Hidden nodes:** Nodes that cannot sense transmissions but cause interference at the receiver

1.3 Simulation Layer (ns-3)

This layer performs packet-level discrete event simulation.

Protocol Implementation:

- **NR-U Nodes:**
  - Use Category-4 Listen Before Talk (LBT)
  - Perform channel sensing, backoff, and retransmissions

- **WiFi Nodes:**
  - Use Distributed Coordination Function (DCF)
  - Based on CSMA/CA mechanism
  - Include DIFS/SIFS timing and contention window handling

Channel Contention Mechanism:

1. Nodes sense the channel
2. If busy, they perform backoff
3. When the backoff counter reaches zero, transmission occurs
4. Collisions may occur due to hidden nodes

1.4 Hidden Node Modeling

Hidden nodes are explicitly modeled based on spatial relationships:

- Nodes outside sensing range but within interference range cause collisions
- This leads to packet loss and performance degradation

1.5 Data Capture Layer (Wireshark)

This layer enables packet-level validation.

Functions:

- Capture packet transmissions and retransmissions
- Record timing information and protocol behavior

Purpose:

- Validate correctness of ns-3 simulation
- Ensure realistic protocol behavior

1.6 Analysis Layer (MATLAB)

This layer processes simulation outputs to compute performance metrics.

Inputs:

- Trace files generated from ns-3 simulations

Metrics Computed:

- **Throughput:** Number of successfully transmitted bits per unit time
- **Packet Delay:** Time from channel contention to successful transmission

Graph Generation:

- Throughput vs packet arrival rate
- Delay vs node density
- Performance vs sensing distance

1.7 Data Flow Architecture

The system follows a sequential processing pipeline:

$$\text{MATLAB Analysis} \rightarrow \text{ns-3 Simulation} \rightarrow \text{Wireshark Capture} \rightarrow \text{Topology Generation} \rightarrow \text{Results}$$

1.8 Key Architectural Features

- **Hybrid Simulation Approach:** Combines ns-3 simulation, MATLAB analysis, and Wireshark validation
- **Realistic Modeling:** Uses random spatial deployment with hidden node effects

- **Scalability:** Supports variation in node density and traffic parameters
- **Modularity:** Each layer operates independently for flexibility

1.9 Summary

The architecture provides a comprehensive framework to analyze the coexistence of NR-U and WiFi systems. By integrating simulation, validation, and analytical processing, the system enables accurate evaluation of throughput, delay, and hidden node effects in wireless networks.architecture diagrams, components, overview

2. Methodology

The methodology adopted in this work follows a structured simulation-based approach to analyze the coexistence of NR-U and WiFi systems in the presence of hidden nodes. The overall process integrates stochastic network modeling, packet-level simulation, and post-processing analysis.

2.1 Modeling Approach

The network is modeled as a spatial wireless system where NR-U base stations and WiFi access points are distributed over a two-dimensional region. The node locations follow independent homogeneous Poisson Point Processes (PPP), enabling realistic representation of random deployments in unlicensed spectrum environments.

The number of nodes is generated as:

$$N_{\text{NRU}} \sim \text{Poisson}(\lambda_{\text{NRU}} A), \quad N_{\text{WiFi}} \sim \text{Poisson}(\lambda_{\text{WiFi}} A)$$

where A represents the deployment area and λ denotes node density.

Each transmitter is paired with a receiver placed randomly within its transmission range. This setup allows the modeling of both communication links and interference relationships.

2.2 Hidden Node Modeling

Hidden node behavior is incorporated using the relationship between sensing range and transmission range. A node is classified as:

- **Sensible node:** if it lies within the sensing range
- **Hidden node:** if it lies outside sensing range but within interference range

This spatial condition results in collisions at the receiver even when the transmitter senses the channel as idle, which is a key factor affecting performance.

2.3 Simulation Framework

The system is implemented using the NS-3 discrete-event simulator. Both NR-U and WiFi systems are modeled using the WiFi PHY/MAC abstraction with distinct parameter configurations.

Channel Access Mechanisms:

- NR-U uses Category-4 Listen Before Talk (LBT)
- WiFi uses Distributed Coordination Function (DCF)

Both mechanisms follow a contention-based access procedure involving:

1. Channel sensing
2. Backoff countdown
3. Transmission upon counter expiry
4. Retransmission in case of collision

2.4 Traffic Modeling

Traffic generation follows a stochastic process to capture realistic network behavior. Packet arrivals are modeled using an exponential inter-arrival distribution, corresponding to a Poisson arrival process.

UDP-based applications are used to generate traffic between transmitter-receiver pairs. The packet arrival rate is varied to analyze both unsaturated and saturated network conditions.

2.5 Physical and MAC Layer Configuration

The simulation uses a shared wireless channel operating in the 5 GHz band. Signal propagation is modeled using the Friis free-space model.

The sensing behavior is controlled using a CCA threshold derived from the sensing distance:

$$P_{\text{CCA}} = P_t - 20 \log_{10} \left(\frac{4\pi d}{\lambda} \right)$$

Both NR-U and WiFi nodes use contention-based MAC parameters including contention window size, AIFS, and backoff stages. These parameters are configured to emulate realistic channel access behavior.

2.6 Performance Metrics

The system performance is evaluated using the following metrics:

Throughput:

$$T = \frac{8B}{t_{\text{meas}} \times 10^6 \times N}$$

where B is the total received data, t_{meas} is the observation time, and N is the number of nodes.

Packet Delay:

$$D = \frac{L}{T \times 10^6} \times 1000$$

where L is the packet size in bits.

2.7 Parameter Variation and Experiments

To analyze system behavior, multiple simulation runs are conducted by varying key parameters:

- Packet arrival rate
- Node density
- Sensing distance
- Transmission rate

Each configuration is executed multiple times using different random seeds to ensure statistical reliability.

2.8 Data Collection and Analysis

Simulation outputs are collected using trace callbacks and stored as structured data. These results are processed using MATLAB and Python scripts to compute performance metrics and generate graphs.

Additionally, packet-level traces are captured using Wireshark to validate the correctness of the simulated protocol behavior.

2.9 Summary

The proposed methodology combines stochastic modeling, discrete-event simulation, and empirical analysis to evaluate the coexistence performance of NR-U and WiFi systems. This approach enables a detailed understanding of the impact of hidden nodes and network parameters on throughput and delay.

3. Implementation Setup

The implementation was carried out in two stages. First, MATLAB was used only as a mathematical modelling environment to validate the analytical behaviour of NR-U and WiFi coexistence. This MATLAB model represents the system using equations and parameter sweeps, but it does not create an actual packet-level network. The main network-level implementation was then developed in NS-3, where nodes, wireless channels, MAC behaviour, packet transmissions, random traffic arrivals, and performance measurements were simulated explicitly.

The NS-3 simulation code was implemented in C++ in the file nru-wifi-coex.cc. The simulation models the coexistence of NR-U and WiFi systems in the unlicensed 5 GHz band. Since the objective was to study coexistence behaviour, both NR-U and WiFi were mapped onto NS-3 WiFi PHY/MAC abstractions while using separate configuration parameters for NR-U and WiFi, such as sensing distance, transmission distance, node density, packet size scaling, and equivalent data rate.

The following NS-3 modules were used:

- `core-module`: simulation time control, command-line arguments, random seed management, and event scheduling.
- `network-module`: creation of nodes, packets, devices, and network containers.
- `mobility-module`: fixed node placement using `ConstantPositionMobilityModel`.
- `wifi-module`: PHY, MAC, channel access, EDCA, CCA threshold, and WiFi device configuration.
- `internet-module`: IPv4 stack installation and IP address assignment.

- `applications-module`: UDP traffic generation using `OnOffApplication` and reception using `PacketSink`.
- `propagation-loss-model`: Friis propagation model for wireless signal attenuation.
- `netanim-module`: included for topology animation support.

The simulation area is defined as a square deployment region of side length `areaSize`, with a default value of 1000 m. NR-U base stations and WiFi access points are spatially distributed according to independent Poisson point processes. The number of NR-U and WiFi transmitter nodes is generated using

$$N_{\text{NRU}} \sim \text{Poisson}(\lambda_{\text{NRU}} A), \qquad N_{\text{WiFi}} \sim \text{Poisson}(\lambda_{\text{WiFi}} A),$$

where A is the deployment area and λ_{NRU}, λ_{WiFi} are the respective node densities. At least one NR-U and one WiFi node are always created to ensure a valid simulation run.

Each NR-U transmitter is paired with one NR-U receiver, and each WiFi access point is paired with one WiFi station. The transmitter position is chosen uniformly inside the simulation region. The receiver is then placed randomly within a circular region around its transmitter, using the configured transmission distance. This is done separately for NR-U and WiFi using `nruTxDist` and `wifiTxDist`. All nodes are static, so the simulation focuses on coexistence and channel access effects rather than mobility.

Both NR-U and WiFi operate on a shared wireless channel created using `YansWifiChannelHelper`. The channel uses:

- `ConstantSpeedPropagationDelayModel` for propagation delay,
- `FriisPropagationLossModel` at 5.18 GHz,
- a common shared channel object for both NR-U and WiFi devices.

The CCA energy detection threshold is derived from the desired sensing distance. The function `SensingDistToCcaThreshold()` converts a sensing distance into a received power threshold using the free-space path loss equation:

$$P_{\text{CCA}} = P_t - 20 \log_{10} \left(\frac{4\pi d}{\lambda} \right),$$

where $P_t = 20$ dBm, d is the sensing distance, and λ is the wavelength at 5.18 GHz. Separate CCA thresholds are computed for NR-U and WiFi using `nruSenseDist` and `wifiSenseDist`. These thresholds are then applied to `CcaEdThreshold` and `RxSensitivity` in the PHY layer.

The PHY configuration uses a transmission power of 20 dBm for both systems. The base PHY rate is set to 54 Mbps using `OfdmRate54Mbps`, while the control rate is set to `OfdmRate6Mbps`. The WiFi standard used is `WIFI_STANDARD_80211a`, which is suitable for modelling operation in the 5 GHz band. NR-U is represented using an equivalent WiFi PHY abstraction, with its effective rate controlled separately through the parameter `nruDataRate`.

The MAC layer uses `AdhocWifiMac` with QoS enabled. For both NR-U and WiFi, the best-effort EDCA access category is configured with:

- minimum contention window `MinCw = 15`,
- maximum contention window `MaxCw = 1023`,
- AIFS number `Aifsn = 2`,
- TXOP limit of 0 microseconds.

RTS/CTS is disabled in the main simulation by setting `RtsCtsThreshold` to a large value.

After the PHY and MAC devices are installed, the Internet stack is installed on all NR-U and WiFi nodes. Each transmitter-receiver pair is assigned a separate IPv4 subnet. NR-U links use addresses beginning with

`10.1.x.x`, while WiFi links use addresses beginning with `10.2.x.x`. This allows the two traffic classes to be separated during measurement.

Traffic is generated using UDP applications. Each receiver runs a `PacketSinkApplication`, while each transmitter runs an `OnOffApplication`. The packet arrival rate is controlled by the command-line parameter `arrivalRate`, expressed in packets per millisecond. This is converted into packets per second inside the code. The inter-arrival time is modelled using an exponential random variable in the main NS-3 simulation, which creates bursty random packet arrivals.

WiFi packets use a packet size of 1500 bytes. NR-U packet size is scaled according to the configured NR-U equivalent data rate:

$$L_{\text{NRU}} = L_{\text{WiFi}} \left(\frac{R_{\text{PHY}}}{R_{\text{NRU}}} \right),$$

where $L_{\text{WiFi}} = 1500$ bytes, $R_{\text{PHY}} = 54$ Mbps, and R_{NRU} is the NR-U equivalent data rate. This scaling allows the simulation to represent different NR-U rates while still using the NS-3 WiFi PHY abstraction.

The simulation exposes the following command-line parameters:

- `arrivalRate`: packet arrival rate in packets/ms,
- `runId`: random run index for repeatability,
- `simTime`: simulation duration in seconds,
- `nruDensity`: NR-U node density,
- `wifiDensity`: WiFi node density,
- `nruSenseDist`: NR-U sensing distance,
- `wifiSenseDist`: WiFi sensing distance,
- `nruTxDist`: NR-U transmission distance,
- `wifiTxDist`: WiFi transmission distance,

- `nruDataRate`: NR-U equivalent data rate,
- `wifiDataRate`: WiFi data rate,
- `areaSize`: side length of the simulation region.

Packet reception is measured using NS-3 trace callbacks connected to the `Rx` event of each packet sink. Separate global statistics vectors are maintained for NR-U and WiFi. Each received packet updates the number of received bytes and received packets for the corresponding system. After the simulation ends, the code computes average per-node throughput as:

$$T = \frac{8B}{t_{\text{meas}} \times 10^6 \times N},$$

where B is the total number of received bytes, t_{meas} is the measurement time, and N is the number of nodes of that technology. For NR-U, the throughput is additionally scaled by the ratio between the NR-U equivalent rate and the base PHY rate.

Delay is estimated from the packet size and measured throughput:

$$D = \frac{L}{T \times 10^6} \times 1000,$$

where L is the packet size in bits and T is the measured throughput in Mbps. The program prints the final metrics as:

- `NRU_THROUGHPUT`,
- `NRU_DELAY`,
- `WIFI_THROUGHPUT`,
- `WIFI_DELAY`,
- `NRU_NODES`,
- `WIFI_NODES`.

Shell scripts were used to automate repeated NS-3 runs for different parameter sweeps. Python plotting scripts were then used to read the generated CSV result files and create the final throughput and delay graphs. The experiments include variation with respect to NR-U rate, sensing distance, node density, and arrival rate.

A separate `NS-3_Changed_Simulation_(Network_Environment)` folder was also maintained for a modified version of the simulation. This changed version keeps the same general NS-3 setup but adds extra MAC-level behaviour such as an RTS/CTS overhead approximation and a WiFi blocking flag around acknowledged MPDU events. We also changed the "OFF" time between packets from packets initially determined by a Poisson formula to a constant, deterministic time interval between packet transmissions.

We present the `nru-wifi-coex.cc` code below.

```
1
2  /*
3  CS255 Mini Project
4  Aadharsh Venkat - 241CS101
5  Amisha Vidya Diwakar - 241CS107
6
7  Program Title: NS-3 (Network Simulator) for NRU Wifi Coexistence
8
9  NOTE: To be placed in ns-3.xx/scratch
10 */
11
12 // NS-3 Toolkits
13
14 #include "ns3/netanim-module.h"
15 #include "ns3/applications-module.h"
16 #include "ns3/core-module.h"
17 #include "ns3/internet-module.h"
18 #include "ns3/mobility-module.h"
19 #include "ns3/network-module.h"
20 #include "ns3/propagation-loss-model.h"
21 #include "ns3/wifi-module.h"
22
23 // C++ Header files
24
25 #include <cmath>
26 #include <iostream>
27 #include <random>
```

```cpp
#include <sstream>
#include <string>
#include <vector>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("NruWifiCoex");

struct NodeStats
{
    uint64_t rxBytes{0};
    uint64_t rxPackets{0};
};
std::vector<NodeStats> g_nruStats;
std::vector<NodeStats> g_wifiStats;

void NruRxCallback(uint32_t idx, Ptr<const Packet> pkt, const Address&)
{
    g_nruStats[idx].rxBytes   += pkt->GetSize();
    g_nruStats[idx].rxPackets += 1;
}

void WifiRxCallback(uint32_t idx, Ptr<const Packet> pkt, const Address
    &)
{
    g_wifiStats[idx].rxBytes   += pkt->GetSize();
    g_wifiStats[idx].rxPackets += 1;
}

double SensingDistToCcaThreshold(double distMeters)
{
    const double freq   = 5.18e9;
    const double c      = 3e8;
    const double lambda = c / freq;
    const double txPow  = 20.0;
    double fspl = 20.0 * std::log10(4.0 * M_PI * distMeters / lambda);
    return txPow - fspl;
}

int main(int argc, char* argv[])
{
    double   arrivalRate  = 3.0;
    uint32_t runId        = 0;
    double   simTime      = 5.0;
    double   nruDensity   = 0.0001;
    double   wifiDensity  = 0.0001;
```

```
    double   nruSenseDist = 100.0;
    double   nruTxDist    = 80.0;
    double   wifiSenseDist= 90.0;
    double   wifiTxDist   = 70.0;
    double   nruDataRate  = 100.0;
    double   wifiDataRate = 54.0;
    double   areaSize     = 1000.0;

    CommandLine cmd;
    cmd.AddValue("arrivalRate",   "Packet arrival rate (packets/ms)",
        arrivalRate);
    cmd.AddValue("runId",         "RNG run index",
        runId);
    cmd.AddValue("simTime",       "Simulation time (s)",
        simTime);
    cmd.AddValue("nruDensity",    "NR-U gNB density (nodes/m2)",
        nruDensity);
    cmd.AddValue("wifiDensity",   "WiFi AP density (nodes/m2)",
        wifiDensity);
    cmd.AddValue("nruSenseDist",  "NR-U sensing distance (m)",
        nruSenseDist);
    cmd.AddValue("nruTxDist",     "NR-U tx distance (m)",
        nruTxDist);
    cmd.AddValue("wifiSenseDist", "WiFi sensing distance (m)",
        wifiSenseDist);
    cmd.AddValue("wifiTxDist",    "WiFi tx distance (m)",
        wifiTxDist);
    cmd.AddValue("nruDataRate",   "NR-U equivalent rate (Mbps)",
        nruDataRate);
    cmd.AddValue("wifiDataRate",  "WiFi channel rate (Mbps)",
        wifiDataRate);
    cmd.AddValue("areaSize",      "Deployment area side (m)",
        areaSize);
    cmd.Parse(argc, argv);

    double nruCcaThreshold  = SensingDistToCcaThreshold(nruSenseDist);
    double wifiCcaThreshold = SensingDistToCcaThreshold(wifiSenseDist);
    static const double TX_POWER_DBM   = 20.0;
    static const double PHY_RATE_MBPS  = 54.0;
    static const double WIFI_PKT_BYTES = 1500.0;
    double nruPktBytes = WIFI_PKT_BYTES * (PHY_RATE_MBPS / nruDataRate)
        ;

    RngSeedManager::SetSeed(42 + runId);
    RngSeedManager::SetRun(runId + 1);
    std::mt19937 rng(42 + runId);
```

```
106     std::uniform_real_distribution<double> uniX(0.0, areaSize);
107     std::uniform_real_distribution<double> uniY(0.0, areaSize);
108
109     double area = areaSize * areaSize;
110     std::poisson_distribution<uint32_t> poisNru (nruDensity  * area);
111     std::poisson_distribution<uint32_t> poisWifi(wifiDensity * area);
112     uint32_t nNru  = std::max(1u, poisNru (rng));
113     uint32_t nWifi = std::max(1u, poisWifi(rng));
114
115     NodeContainer nruAPs,  nruUsers;
116     NodeContainer wifiAPs, wifiUsers;
117     nruAPs.Create(nNru);   nruUsers.Create(nNru);
118     wifiAPs.Create(nWifi); wifiUsers.Create(nWifi);
119
120     MobilityHelper mob;
121     mob.SetMobilityModel("ns3::ConstantPositionMobilityModel");
122     mob.Install(nruAPs);  mob.Install(nruUsers);
123     mob.Install(wifiAPs); mob.Install(wifiUsers);
124
125     std::uniform_real_distribution<double> uniAngle(0.0, 2.0 * M_PI);
126     std::uniform_real_distribution<double> uniUnit (0.0, 1.0);
127
128     for (uint32_t i = 0; i < nNru; i++)
129     {
130         double x = uniX(rng), y = uniY(rng);
131         nruAPs.Get(i)->GetObject<MobilityModel>()->SetPosition(Vector(x
                , y, 0));
132         double r = nruTxDist * std::sqrt(uniUnit(rng));
133         double a = uniAngle(rng);
134         nruUsers.Get(i)->GetObject<MobilityModel>()->SetPosition(
135             Vector(x + r*std::cos(a), y + r*std::sin(a), 0));
136     }
137     for (uint32_t i = 0; i < nWifi; i++)
138     {
139         double x = uniX(rng), y = uniY(rng);
140         wifiAPs.Get(i)->GetObject<MobilityModel>()->SetPosition(Vector(
                x, y, 0));
141         double r = wifiTxDist * std::sqrt(uniUnit(rng));
142         double a = uniAngle(rng);
143         wifiUsers.Get(i)->GetObject<MobilityModel>()->SetPosition(
144             Vector(x + r*std::cos(a), y + r*std::sin(a), 0));
145     }
146
147     YansWifiChannelHelper channel;
148     channel.SetPropagationDelay("ns3::
            ConstantSpeedPropagationDelayModel");
```

```
149     channel.AddPropagationLoss("ns3::FriisPropagationLossModel",
150                                "Frequency", DoubleValue(5.18e9));
151     Ptr<YansWifiChannel> sharedCh = channel.Create();
152
153     YansWifiPhyHelper nruPhy, wifiPhy;
154     nruPhy.SetChannel (sharedCh); wifiPhy.SetChannel(sharedCh);
155     nruPhy.Set ("TxPowerStart",   DoubleValue(TX_POWER_DBM));
156     nruPhy.Set ("TxPowerEnd",     DoubleValue(TX_POWER_DBM));
157     nruPhy.Set ("CcaEdThreshold", DoubleValue(nruCcaThreshold));
158     nruPhy.Set ("RxSensitivity",  DoubleValue(nruCcaThreshold));
159     wifiPhy.Set("TxPowerStart",   DoubleValue(TX_POWER_DBM));
160     wifiPhy.Set("TxPowerEnd",     DoubleValue(TX_POWER_DBM));
161     wifiPhy.Set("CcaEdThreshold", DoubleValue(wifiCcaThreshold));
162     wifiPhy.Set("RxSensitivity",  DoubleValue(wifiCcaThreshold));
163
164     WifiHelper nruWifi, wifiWifi;
165     nruWifi.SetStandard(WIFI_STANDARD_80211a);
166     nruWifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
167                                     "DataMode",    StringValue("
                                           OfdmRate54Mbps"),
168                                     "ControlMode", StringValue("
                                           OfdmRate6Mbps"));
169     wifiWifi.SetStandard(WIFI_STANDARD_80211a);
170     wifiWifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
171                                      "DataMode",    StringValue("
                                            OfdmRate54Mbps"),
172                                      "ControlMode", StringValue("
                                            OfdmRate6Mbps"));
173
174     WifiMacHelper nruMac, wifiMac;
175     nruMac.SetType ("ns3::AdhocWifiMac", "QosSupported", BooleanValue(
            true));
176     wifiMac.SetType("ns3::AdhocWifiMac", "QosSupported", BooleanValue(
            true));
177     Config::SetDefault("ns3::WifiRemoteStationManager::RtsCtsThreshold"
            ,
178                        UintegerValue(65535));
179
180     NetDeviceContainer nruApDev   = nruWifi.Install (nruPhy,  nruMac,
            nruAPs);
181     NetDeviceContainer nruStaDev  = nruWifi.Install (nruPhy,  nruMac,
            nruUsers);
182     NetDeviceContainer wifiApDev  = wifiWifi.Install(wifiPhy, wifiMac,
            wifiAPs);
183     NetDeviceContainer wifiStaDev = wifiWifi.Install(wifiPhy, wifiMac,
            wifiUsers);
```

```
184
185     for (uint32_t i = 0; i < nruApDev.GetN(); i++)
186     {
187         Ptr<QosTxop> edca = DynamicCast<WifiNetDevice>(nruApDev.Get(i))
188                                 ->GetMac()->GetQosTxop(AC_BE);
189         edca->SetMinCw(15); edca->SetMaxCw(1023);
190         edca->SetAifsn(2);  edca->SetTxopLimit(MicroSeconds(0));
191     }
192     for (uint32_t i = 0; i < wifiApDev.GetN(); i++)
193     {
194         Ptr<QosTxop> edca = DynamicCast<WifiNetDevice>(wifiApDev.Get(i)
                )
195                                 ->GetMac()->GetQosTxop(AC_BE);
196         edca->SetMinCw(15); edca->SetMaxCw(1023);
197         edca->SetAifsn(2);  edca->SetTxopLimit(MicroSeconds(0));
198     }
199
200     InternetStackHelper inet;
201     inet.Install(nruAPs);  inet.Install(nruUsers);
202     inet.Install(wifiAPs); inet.Install(wifiUsers);
203
204     Ipv4AddressHelper ipv4;
205     std::vector<Ipv4Address> nruUserAddrs(nNru), wifiUserAddrs(nWifi);
206
207     for (uint32_t i = 0; i < nNru; i++)
208     {
209         std::ostringstream ss;
210         ss << "10.1." << (i/64) << "." << ((i%64)*4);
211         ipv4.SetBase(ss.str().c_str(), "255.255.255.252");
212         NetDeviceContainer p; p.Add(nruApDev.Get(i)); p.Add(nruStaDev.
                Get(i));
213         nruUserAddrs[i] = ipv4.Assign(p).GetAddress(1);
214     }
215     for (uint32_t i = 0; i < nWifi; i++)
216     {
217         std::ostringstream ss;
218         ss << "10.2." << (i/64) << "." << ((i%64)*4);
219         ipv4.SetBase(ss.str().c_str(), "255.255.255.252");
220         NetDeviceContainer p; p.Add(wifiApDev.Get(i)); p.Add(wifiStaDev
                .Get(i));
221         wifiUserAddrs[i] = ipv4.Assign(p).GetAddress(1);
222     }
223
224     g_nruStats.resize (nNru);
225     g_wifiStats.resize(nWifi);
226
```

```
227     uint16_t port       = 9;
228     double   arrPerSec  = arrivalRate * 1000.0;
229     double   meanIAT    = 1.0 / arrPerSec;
230     double   phyRateBps = PHY_RATE_MBPS * 1e6;
231
232     for (uint32_t i = 0; i < nNru; i++)
233     {
234         PacketSinkHelper sink("ns3::UdpSocketFactory",
235                               InetSocketAddress(Ipv4Address::GetAny(),
                                      port));
236         ApplicationContainer sinkApp = sink.Install(nruUsers.Get(i));
237         sinkApp.Start(Seconds(0.0)); sinkApp.Stop(Seconds(simTime));
238         Ptr<PacketSink> sinkPtr = DynamicCast<PacketSink>(sinkApp.Get
                (0));
239
240         double onT  = (nruPktBytes * 8.0) / phyRateBps;
241         double offM = std::max(1e-9, meanIAT - onT);
242
243         OnOffHelper onoff("ns3::UdpSocketFactory",
244                           InetSocketAddress(nruUserAddrs[i], port));
245         onoff.SetConstantRate(DataRate(static_cast<uint64_t>(phyRateBps
                )));
246         onoff.SetAttribute("PacketSize", UintegerValue((uint32_t)
                nruPktBytes));
247         onoff.SetAttribute("OnTime",  StringValue(
248             "ns3::ConstantRandomVariable[Constant=" + std::to_string(
                    onT) + "]"));
249         onoff.SetAttribute("OffTime", StringValue(
250             "ns3::ExponentialRandomVariable[Mean=" + std::to_string(
                    offM) + "]"));
251
252         ApplicationContainer src = onoff.Install(nruAPs.Get(i));
253         src.Start(Seconds(0.1)); src.Stop(Seconds(simTime));
254
255         uint32_t idx = i;
256         sinkPtr->TraceConnectWithoutContext("Rx",
257             MakeBoundCallback(&NruRxCallback, idx));
258     }
259
260     for (uint32_t i = 0; i < nWifi; i++)
261     {
262         PacketSinkHelper sink("ns3::UdpSocketFactory",
263                               InetSocketAddress(Ipv4Address::GetAny(),
                                      port+1));
264         ApplicationContainer sinkApp = sink.Install(wifiUsers.Get(i));
265         sinkApp.Start(Seconds(0.0)); sinkApp.Stop(Seconds(simTime));
```

```
266         Ptr<PacketSink> sinkPtr = DynamicCast<PacketSink>(sinkApp.Get
                (0));
267
268         double onT  = (WIFI_PKT_BYTES * 8.0) / phyRateBps;
269         double offM = std::max(1e-9, meanIAT - onT);
270
271         OnOffHelper onoff("ns3::UdpSocketFactory",
272                           InetSocketAddress(wifiUserAddrs[i], port+1));
273         onoff.SetConstantRate(DataRate(static_cast<uint64_t>(phyRateBps
                )));
274         onoff.SetAttribute("PacketSize", UintegerValue((uint32_t)
                WIFI_PKT_BYTES));
275         onoff.SetAttribute("OnTime",  StringValue(
276             "ns3::ConstantRandomVariable[Constant=" + std::to_string(
                    onT) + "]"));
277         onoff.SetAttribute("OffTime", StringValue(
278             "ns3::ExponentialRandomVariable[Mean=" + std::to_string(
                    offM) + "]"));
279
280         ApplicationContainer src = onoff.Install(wifiAPs.Get(i));
281         src.Start(Seconds(0.1)); src.Stop(Seconds(simTime));
282
283         uint32_t idx = i;
284         sinkPtr->TraceConnectWithoutContext("Rx",
285             MakeBoundCallback(&WifiRxCallback, idx));
286
287     }
288
289     // Flow monitor also provided
290
291     /*
292     FlowMonitorHelper flowmon;
293     Ptr<FlowMonitor> monitor = flowmon.InstallAll();
294     */
295
296     Simulator::Stop(Seconds(simTime));
297     Simulator::Run();
298
299     /*
300     monitor->CheckForLostPackets();
301     Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier
            >(flowmon.GetClassifier());
302     std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->
            GetFlowStats();
303
304     double totalDelayNru = 0.0;
```

```
305     double totalDelayWifi = 0.0;
306     uint32_t rxPacketsNru = 0;
307     uint32_t rxPacketsWifi = 0;
308
309     for (std::map<FlowId, FlowMonitor::FlowStats>::const_iterator i =
            stats.begin(); i != stats.end(); ++i)
310     {
311         Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(i->first
                );
312
313         if (t.destinationPort == port) {
314             totalDelayNru += i->second.delaySum.GetSeconds();
315             rxPacketsNru += i->second.rxPackets;
316         }
317         else if (t.destinationPort == port + 1) {
318             totalDelayWifi += i->second.delaySum.GetSeconds();
319             rxPacketsWifi += i->second.rxPackets;
320         }
321     }
322
323     double actualNruDly = (rxPacketsNru > 0) ? (totalDelayNru /
            rxPacketsNru) * 1000.0 : 0.0;
324     double actualWifiDly = (rxPacketsWifi > 0) ? (totalDelayWifi /
            rxPacketsWifi) * 1000.0 : 0.0;
325     */
326
327     Simulator::Destroy();
328
329     double measureTime = simTime - 0.1;
330
331     uint64_t nruBytes = 0;
332     uint64_t nruPkts  = 0;
333     for (auto& s : g_nruStats) { nruBytes += s.rxBytes; nruPkts += s.
            rxPackets; }
334     double nruTp = (nNru > 0 && nruBytes > 0)
335         ? ((double)nruBytes * 8.0 / measureTime / 1e6 / nNru
336            * (nruDataRate / PHY_RATE_MBPS))
337         : 0.0;
338
339     double nruPktBits = nruPktBytes * 8.0 * (nruDataRate /
            PHY_RATE_MBPS);
340     double nruDly = (nruTp > 0)
341         ? (nruPktBits / (nruTp * 1e6) * 1000.0)
342         : 0.0;
343
344     uint64_t wifiBytes = 0;
```

```
345     uint64_t wifiPkts  = 0;
346     for (auto& s : g_wifiStats) { wifiBytes += s.rxBytes; wifiPkts += s
            .rxPackets; }
347     double wifiTp = (nWifi > 0 && wifiBytes > 0)
348         ? ((double)wifiBytes * 8.0 / measureTime / 1e6 / nWifi)
349         : 0.0;
350
351     double wifiPktBits = WIFI_PKT_BYTES * 8.0;
352     double wifiDly = (wifiTp > 0)
353         ? (wifiPktBits / (wifiTp * 1e6) * 1000.0)
354         : 0.0;
355
356     std::cout << "NRU_THROUGHPUT "  << nruTp   << "\n";
357     std::cout << "NRU_DELAY "       << nruDly  << "\n";
358     std::cout << "WIFI_THROUGHPUT " << wifiTp  << "\n";
359     std::cout << "WIFI_DELAY "      << wifiDly << "\n";
360     std::cout << "NRU_NODES "       << nNru    << "\n";
361     std::cout << "WIFI_NODES "      << nWifi   << "\n";
362
363     return 0;
364 }
```

4. Results and Explanation

4.1 MATLAB (Mathematical Model) Results

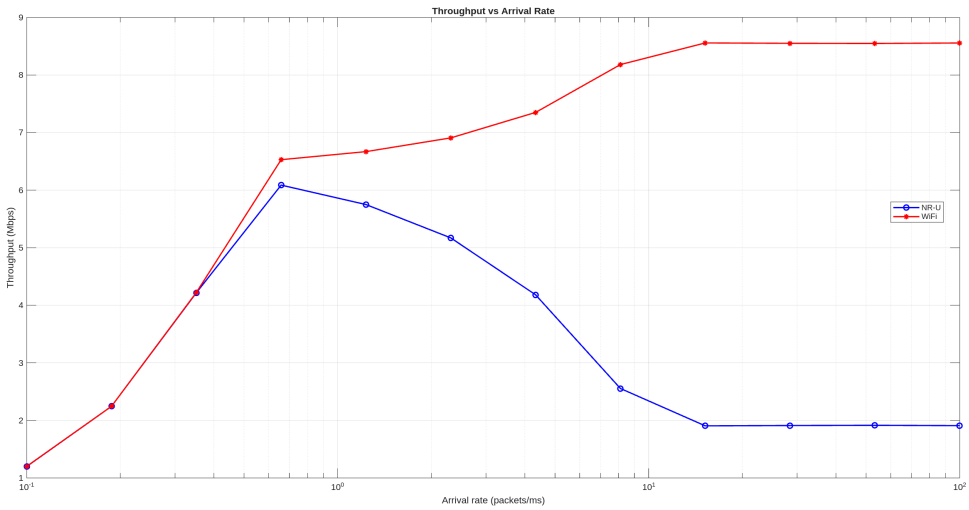

Figure 2: Throughput vs Arrival rate

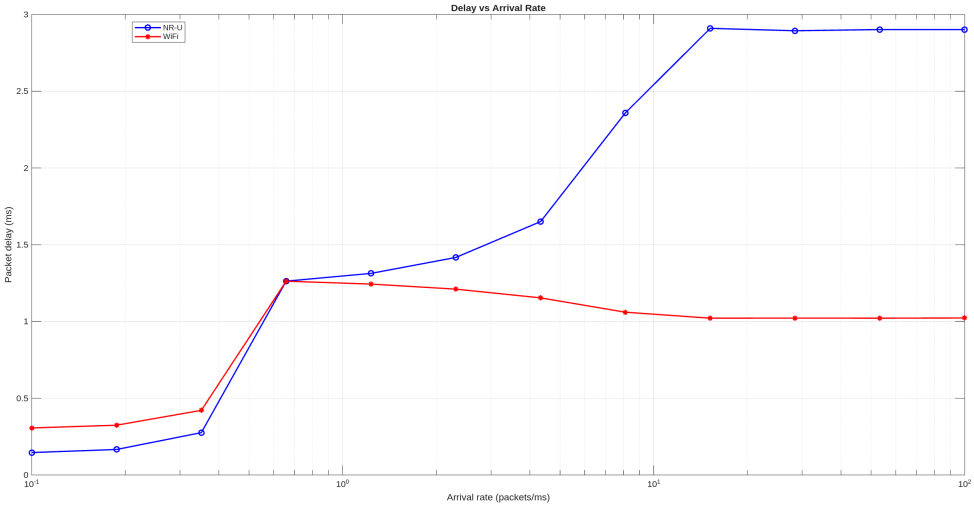

Figure 3: Delay vs Arrival rate

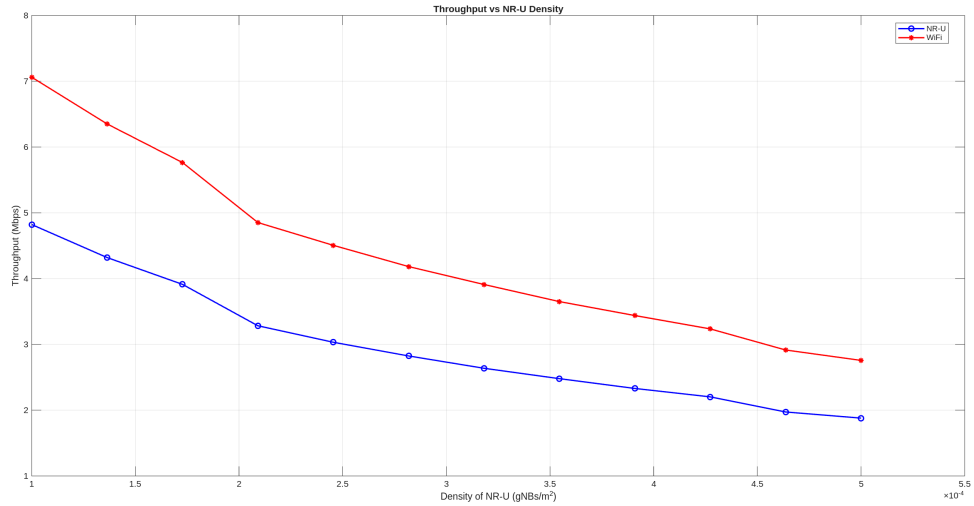

Figure 4: Throughput vs NRU Density

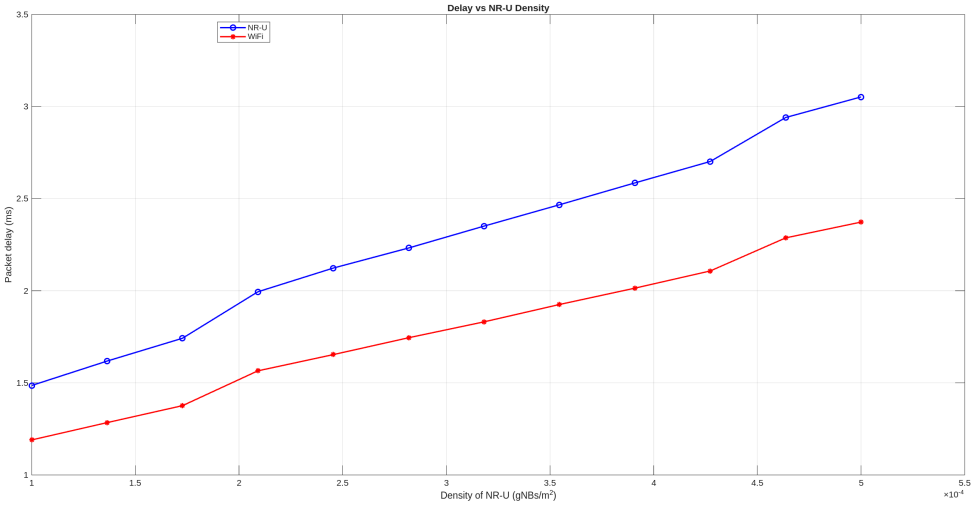

Figure 5: Delay vs NRU Density

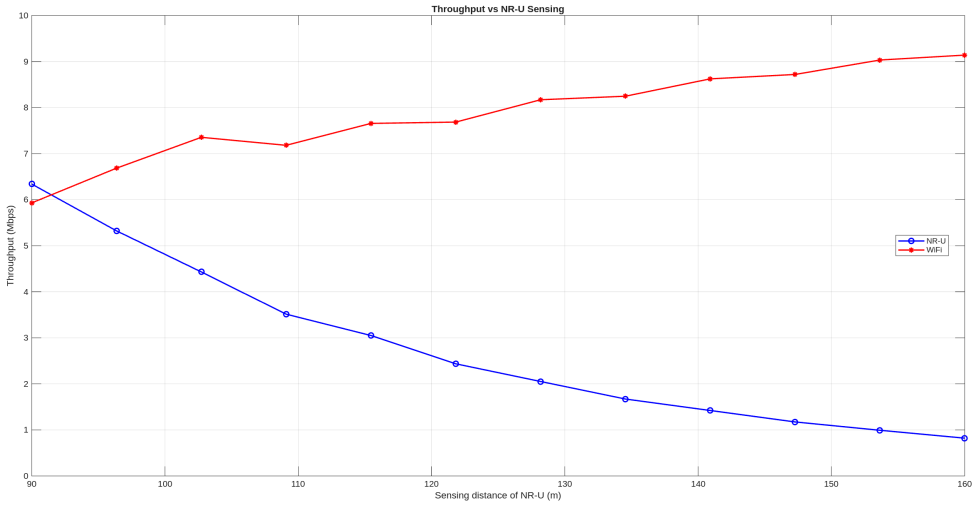

Figure 6: Throughput vs NRU Sensing Distance

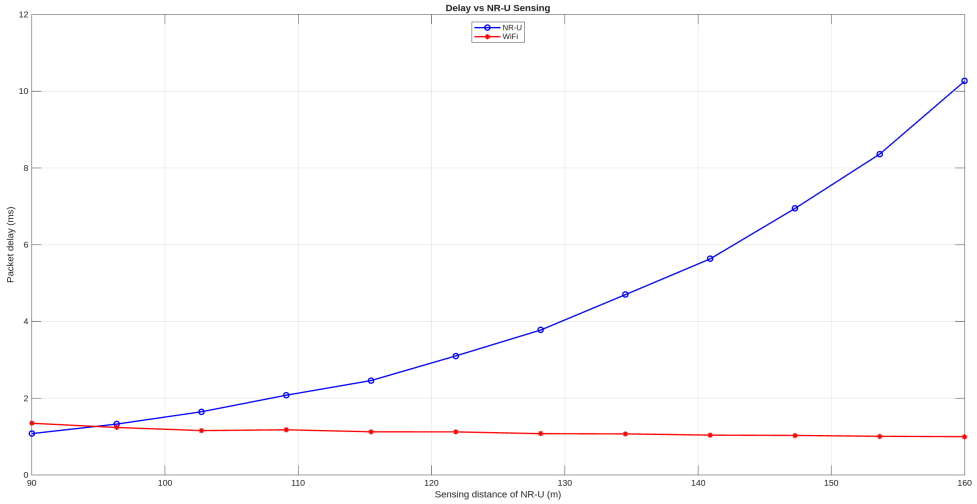

Figure 7: Delay vs NRU Sensing Distance

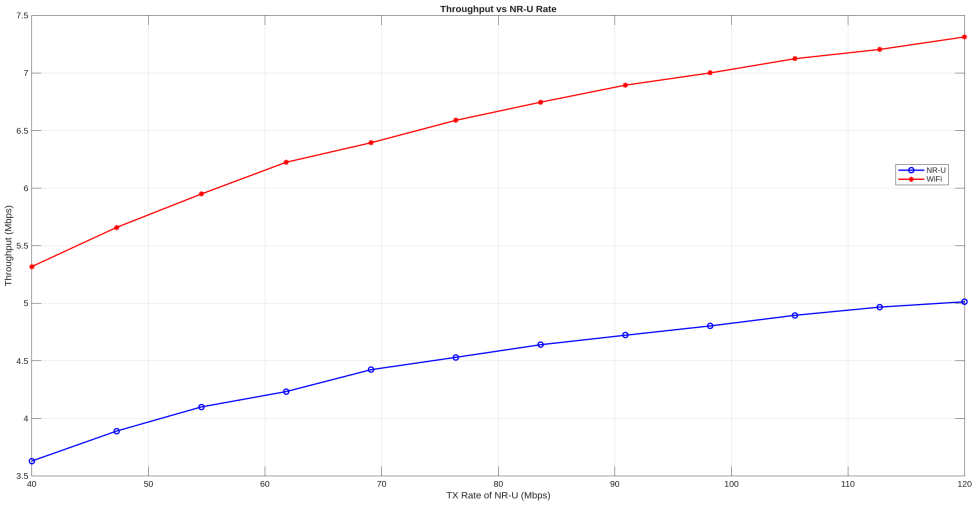

Figure 8: Delay vs NRU Transmission Rate

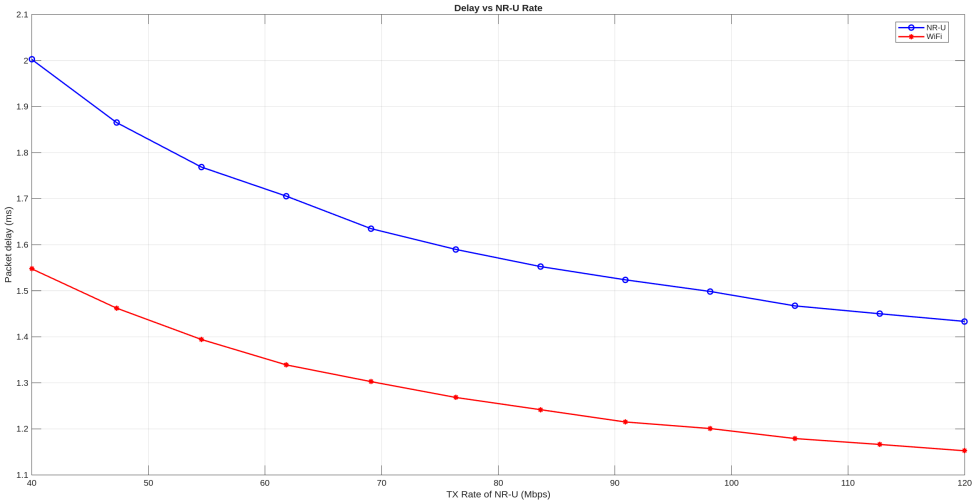

Figure 9: Delay vs NRU Transmission Rate

4.2 NS-3 (Network Simulation)

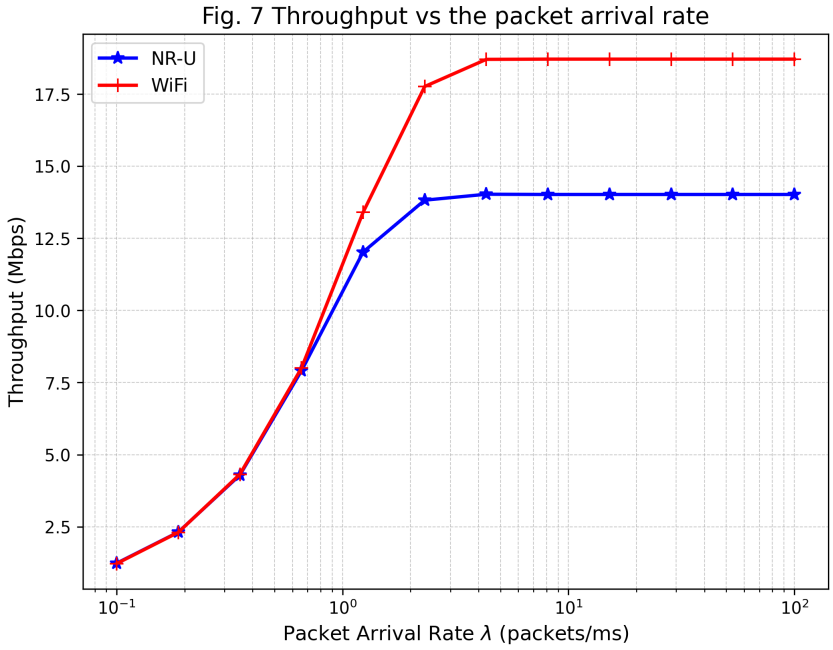

Figure 10: Throughput vs Arrival Rate

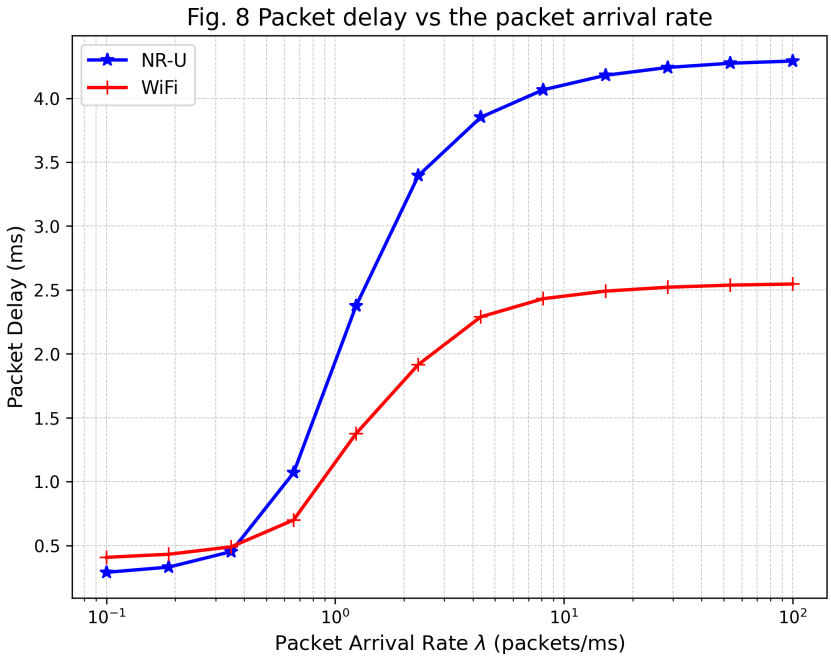

Figure 11: Delay vs Arrival Rate

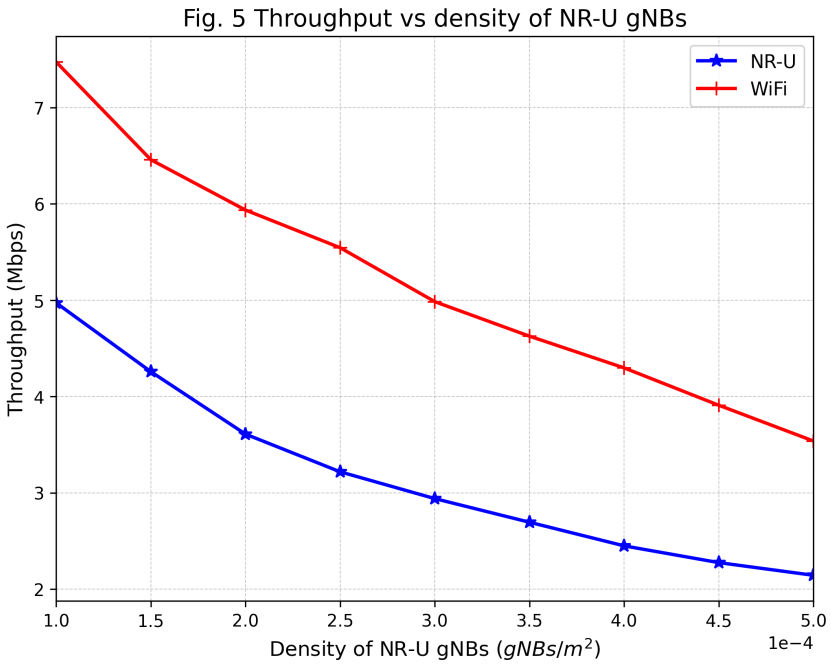

Figure 12: Throughput vs NR-U Density

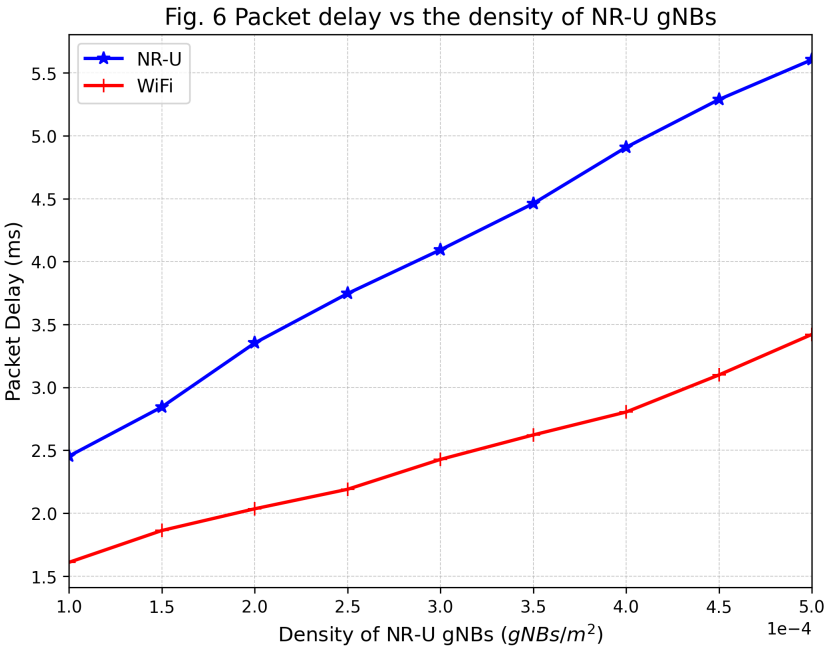

Figure 13: Delay vs NR-U Density

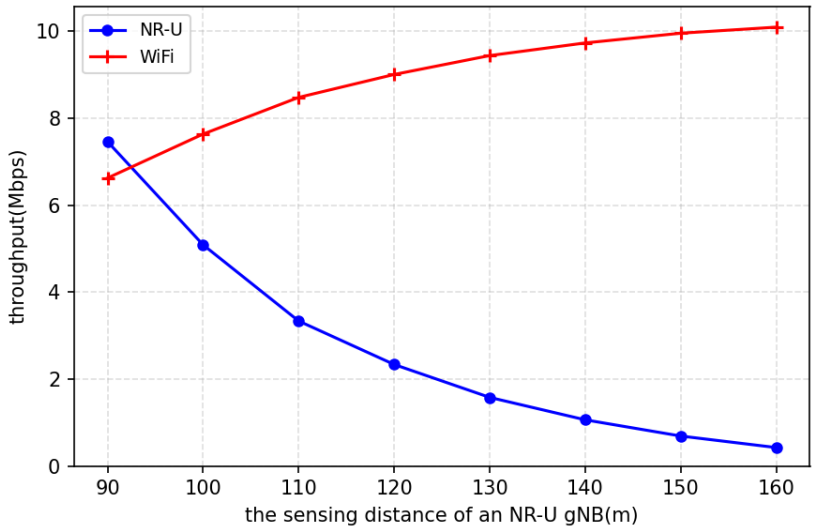

Figure 14: Throughput vs NR-U Sensing Distance

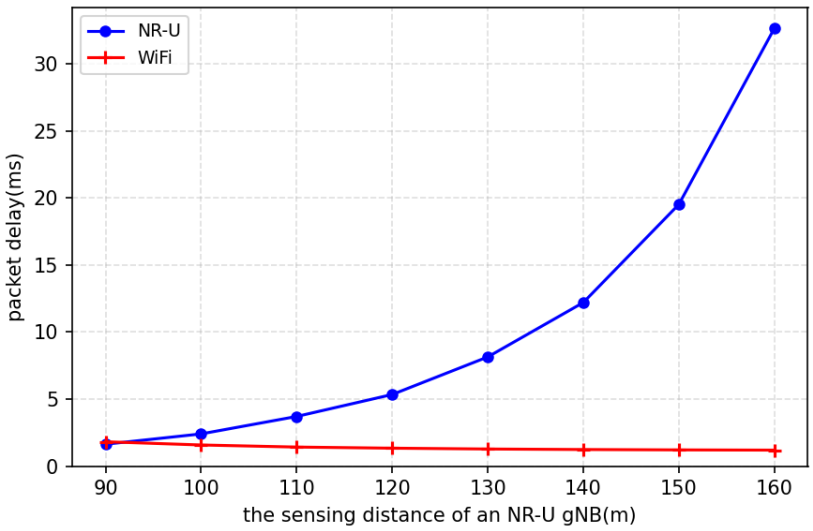

Figure 15: Delay vs NR-U Sensing Distance

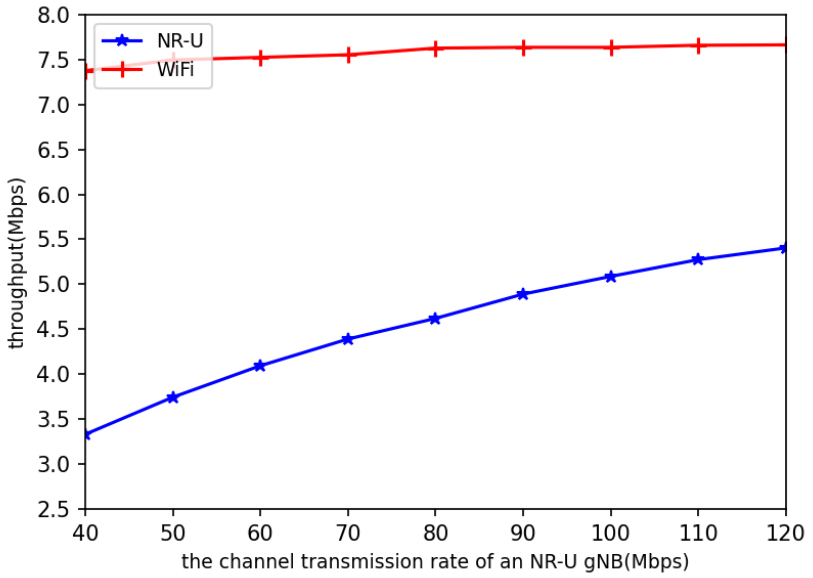

Figure 16: Throughput vs NR-U Transmission Rate

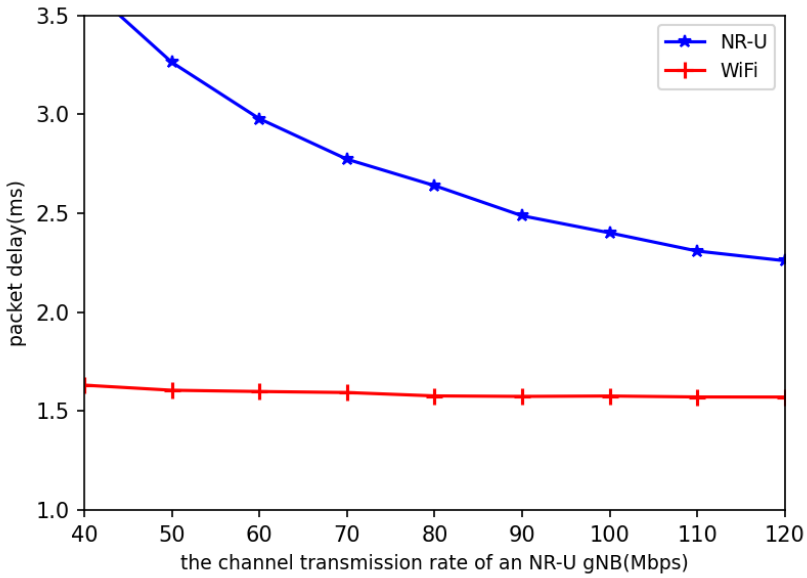

Figure 17: Delay vs NR-U Transmission Rate

4.3 Specific Results on NS-3 Changed Implementation

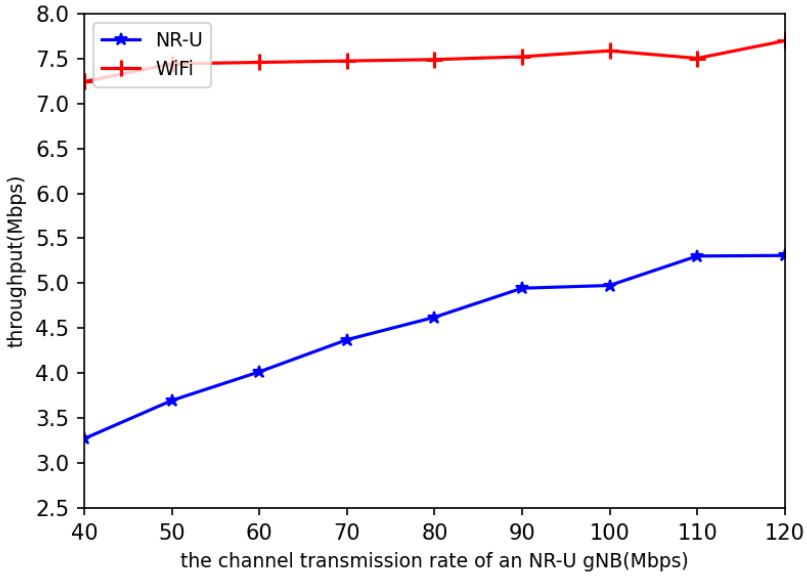

Figure 18: Throughput vs NR-U Transmission Rate (Constant Arrival Rate)

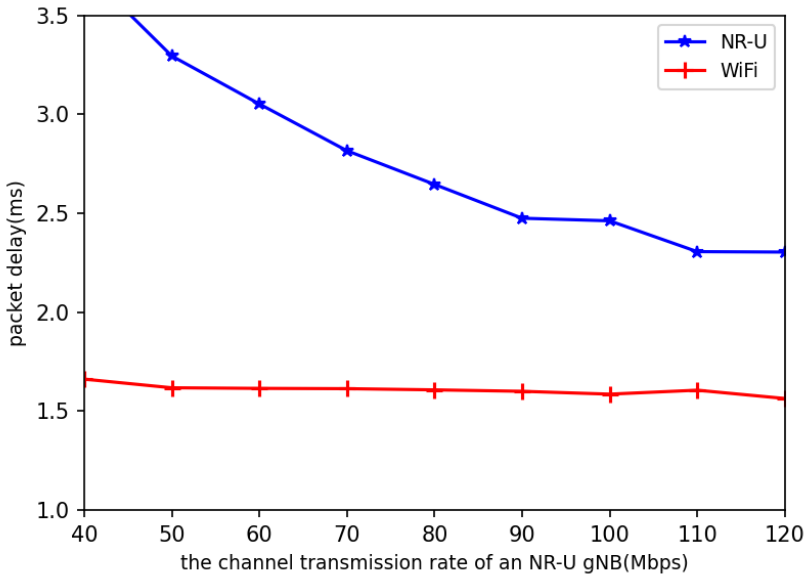

Figure 19: Delay vs NR-U Transmission Rate (Constant Arrival Rate)

4.4 Wireshark Screenshot

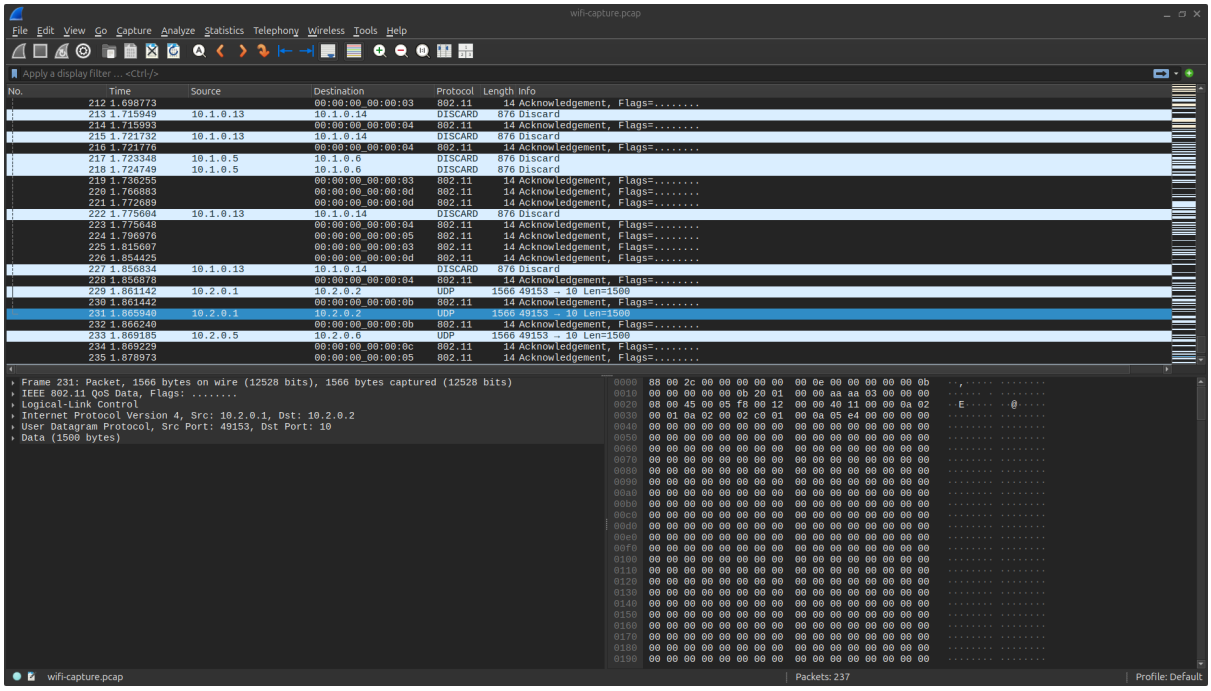

Figure 20: Packets being displayed in WireShark

4.5 Overall Comparison with the Original Paper

Our MATLAB model follows the paper more directly because it implements a mathematical CSMA/event-based abstraction. It uses queue states, contention windows, retransmissions, collision probability, hidden-node probability, packet durations, and sensing-distance effects. Therefore, MATLAB results are closer to the analytical trends of the paper.

The original NS-3 simulation is more network-oriented. It creates actual NR-U and WiFi node pairs, places them randomly in a physical area, installs PHY/MAC devices, assigns IP addresses, generates UDP packets, and measures received bytes through trace callbacks. Because NS-3 models packet-level behaviour, random topology, propagation loss, PHY sensing, and MAC contention, its curves are not expected to match MATLAB exactly point-by-point. However, the major trends should remained consistent.

4.6 MATLAB Mathematical Model Results

The MATLAB results reproduce the mathematical behaviour of the paper. Since MATLAB uses a simplified event model instead of a full packet-level network, the results are smoother and more directly controlled by the equations used in the implementation.

For packet arrival rate, the MATLAB model shows that throughput initially increases as the offered load increases. At low arrival rates, both NR-U and WiFi are underutilized because packets are not generated frequently enough to fully occupy the channel. As the arrival rate increases, the channel becomes busier and throughput improves. At high arrival rates, the channel becomes saturated, contention becomes stronger, and delay increases. This agrees with the paper's expected behaviour: higher traffic load improves utilization at first, but later increases congestion and packet delay.

For NR-U density, MATLAB shows that increasing the density of NR-

U gNBs increases the number of contenders in the same contention region. This causes more backoff, more collisions, and higher delay. NR-U throughput may initially improve due to more active NR-U transmitters, but per-node performance eventually suffers because the channel becomes more congested. WiFi performance also degrades because WiFi shares the same unlicensed channel with a larger number of NR-U transmitters.

For NR-U sensing distance, MATLAB captures the trade-off described in the paper. A larger sensing distance allows NR-U to detect more neighboring transmissions, which can reduce hidden-node collisions. However, it also makes NR-U defer more often. In our MATLAB code, this is represented using a sensing-distance based defer multiplier, which increases the NR-U contention window. As a result, very large sensing distances can reduce NR-U throughput and increase delay.

For NR-U transmission rate, MATLAB shows that increasing the NR-U rate improves NR-U performance because packet transmission time decreases. This allows NR-U packets to occupy the channel for a shorter duration. Consequently, NR-U delay decreases and throughput improves. WiFi is affected indirectly because NR-U channel occupancy changes as the NR-U rate changes.

4.7 Original NS-3 Simulation Results

The original NS-3 simulation confirms the main coexistence behaviour of the paper, but with more realistic packet-level variation. The NS-3 results are averaged over multiple runs, where each run uses a different random seed and a different random spatial deployment.

4.7.1 NR-U Transmission Rate

In the original NS-3 simulation, increasing the NR-U data rate from 40 Mbps to 120 Mbps improves NR-U throughput and reduces NR-U delay. The averaged result changes approximately as follows:

NR-U throughput: 3.331 Mbps $\rightarrow 5.404 Mbps$

NR-U delay: 3.662 ms $\rightarrow$ 2.260ms

This matches the original paper's expected trend. A higher NR-U transmission rate reduces the effective packet transmission time, so NR-U can complete transmissions faster. WiFi throughput remains comparatively stable, changing from about 7.375 Mbps to 7.662 Mbps, because WiFi's own PHY rate is fixed at 54 Mbps. This indicates that the NR-U rate mainly affects NR-U performance, while WiFi is influenced only indirectly through channel occupancy.

4.7.2 NR-U Sensing Distance

When NR-U sensing distance increases from 90 m to 160 m, the original NS-3 simulation shows a major reduction in NR-U throughput:

NR-U throughput: 7.461 Mbps $\rightarrow$ 0.423$Mbps$

At the same time, NR-U delay increases sharply:

NR-U delay: 1.633 ms $\rightarrow$ 32.651ms

This result strongly supports the paper's discussion of sensing-distance trade-offs. A larger sensing distance makes NR-U more conservative. It senses more transmissions in the surrounding area and therefore defers more often. This reduces NR-U's chance to access the channel.

WiFi benefits in this case. WiFi throughput increases from about 6.623 Mbps to 10.094 Mbps, while WiFi delay decreases from about 1.816 ms to 1.193 ms. This happens because NR-U becomes less aggressive as its sensing distance increases, leaving more channel access opportunities for WiFi.

4.7.3 NR-U Density

When NR-U density increases from 1×10^{-4} nodes/m^2 to 5×10^{-4} nodes/m^2, the original NS-3 simulation shows degradation for both systems:

NR-U throughput: 4.972 Mbps $\rightarrow$ 2.143$Mbps$

WiFi throughput: 7.474 Mbps $\rightarrow 3.537 Mbps$

NR-U delay increases from about 2.454 ms to 5.605 ms, while WiFi delay increases from about 1.609 ms to 3.421 ms. This agrees with the original paper: increasing NR-U density increases the number of transmitters competing for the same channel. More contention leads to more backoff, more channel access delay, and lower per-node throughput. Since WiFi shares the same unlicensed medium, WiFi also suffers when NR-U density becomes high.

4.7.4 Packet Arrival Rate

For packet arrival rate, the original NS-3 simulation shows that throughput increases as arrival rate increases:

NR-U throughput: 1.185 Mbps $\rightarrow 5.114 Mbps$

WiFi throughput: 1.177 Mbps $\rightarrow 7.479 Mbps$

This is expected because at low arrival rates, the channel is lightly loaded and many transmission opportunities are unused. As the arrival rate increases, more packets are available for transmission, so measured throughput increases.

However, delay also increases at high load:

NR-U delay: 0.773 ms $\rightarrow 7.891 ms$

WiFi delay: 0.879 ms $\rightarrow 5.639 ms$

This agrees with the paper's saturation behaviour. Higher packet arrival rate improves channel utilization, but it also creates more queueing, contention, and waiting time.

4.8 Changed NS-3 Simulation Results

The changed NS-3 simulation gives results close to the original NS-3 simulation, but with small differences caused by the additional MAC-level

changes. The changed version introduces an RTS/CTS overhead approximation and WiFi blocking behaviour around acknowledged MPDU events.

For NR-U transmission rate, the changed simulation gives:

NR-U throughput: 3.272 Mbps $\rightarrow 5.308 Mbps$

NR-U delay: 3.733 ms $\rightarrow 2.304 ms$

This is very close to the original NS-3 trend. NR-U throughput still increases as NR-U rate increases, and NR-U delay still decreases. WiFi throughput remains around 7.2–7.7 Mbps, showing that the changed MAC behaviour does not reverse the main conclusion.

For NR-U sensing distance, the changed simulation gives the same trend as the original NS-3 simulation. NR-U throughput drops significantly as sensing distance increases, while WiFi throughput improves. This confirms that sensing distance is one of the most important coexistence parameters. A larger NR-U sensing distance protects WiFi more, but it severely reduces NR-U access opportunities.

For NR-U density, the changed simulation again follows the same broad trend. NR-U throughput decreases from about 4.976 Mbps to 2.182 Mbps as density increases. WiFi throughput decreases from about 7.585 Mbps to 3.800 Mbps. Delay increases for both systems. Compared with the original NS-3 result, the changed version gives slightly better WiFi throughput at high density, which is consistent with the added MAC-level blocking/overhead making WiFi behaviour somewhat more protected in the coexistence environment.

4.9 Comparison Between MATLAB and NS-3

The MATLAB and NS-3 results agree qualitatively, but they are not identical numerically. This is expected because the MATLAB model is a mathematical abstraction, while NS-3 is a packet-level network simulator.

The MATLAB model directly controls contention behaviour through

formulas for collision probability, hidden-node probability, event duration, contention windows, and sensing penalties. Therefore, MATLAB curves are smoother and more idealized. It is better for understanding the analytical trends from the original paper.

The NS-3 model includes more implementation details: actual nodes, actual packet generation, random spatial placement, PHY thresholds, propagation loss, IP addressing, UDP applications, and trace-based reception measurement. Because of this, the NS-3 graphs include effects that the MATLAB model abstracts away. For example, random node placement can change the number of effective interferers from run to run, and PHY-layer sensing thresholds can make channel access behaviour more sensitive to distance.

Despite these differences, both MATLAB and NS-3 support the same conclusions as the paper:

- Increasing packet arrival rate increases throughput until the channel becomes congested, but it also increases delay.
- Increasing NR-U density increases contention and reduces coexistence performance for both NR-U and WiFi.
- Increasing NR-U sensing distance protects WiFi but can significantly reduce NR-U throughput.
- Increasing NR-U transmission rate improves NR-U throughput and reduces NR-U delay.

4.10 Final Inference

Overall, our results are consistent with the original paper at the trend level. The MATLAB model validates the paper's mathematical behaviour, while the original NS-3 simulation shows that the same behaviour appears in a packet-level network simulator. The changed NS-3 simulation further

shows that adding MAC-level overhead modifies the exact values but does not change the main conclusions.

The most important observation is the trade-off between NR-U aggressiveness and WiFi protection. When NR-U uses a smaller sensing distance, it gains more channel access and higher throughput, but WiFi performance can suffer. When NR-U uses a larger sensing distance, WiFi throughput improves, but NR-U delay increases sharply. Similarly, increasing NR-U density harms both systems because the shared unlicensed channel becomes more congested. These results support the original paper's conclusion that fair NR-U/WiFi coexistence depends strongly on sensing configuration, node density, and traffic load.

5. References

References

[1] Q. Ren and J. Zheng, "Simulation-based Performance Analysis of an NR-U and WiFi Coexistence System in the Presence of Hidden Nodes," in *Proceedings of the IEEE International Conference on Computing, Networking and Communications (ICNC)*, 2023.

[2] 3GPP TS 37.213, "Physical layer procedures for shared spectrum channel access," Release 17, 2022.

[3] IEEE Standard 802.11, "Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," IEEE, 2020.

[4] The ns-3 Network Simulator, Available: `https://www.nsnam.org`

[5] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, "Network simulations with the ns-3 simulator," in *Proceedings of the ACM SIGCOMM*, 2008.

[6] Wireshark Network Analyzer, Available: `https://www.wireshark.org`

[7] MATLAB, Version R2023a, The MathWorks Inc., Natick, Massachusetts, USA.

[8] S. Lagen et al., "New radio beam-based access to unlicensed spectrum: Design challenges and solutions," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 8–37, 2020.